Mentimeter

Laboratorul 12

Java features

Mentimeter

Instructions

5

Mentimeter

Interfete

- in mod normal avem numai metode neimplementate (abstracte)
- din Java 8 putem sa avem si metode default (non-statice) si statice

Mentimeter

Metode statice

- ca si metodele statice din clase
- **NU** sunt mostenite de clasa care implementeaza clasa/interfata care extinde interfata

Mentimeter

```
interface Student {

  void print();

  static String[] splitString(String s) {
    return s.split( regex: " ");
  }
}
```

Mentimeter

Metode default

- metode obisnuite, non-statice, publice, marcate prin keywordul default
- **mostenite** de clasele care implementeaza interfata
- pot fi suprascrise in clase

Mentimeter

Implementarea a 2 interfete in acelasi timp

Daca au metode default cu aceeasi semnatura => eroare de compilare => clasa trebuie sa isi faca propria implementare

Mentimeter

```java
interface Human {

  default void print() {
    System.out.println("Human");
  }
}

interface Student {

  default void print() {
    System.out.println("Student");
  }
}

class Me implements Human, Student {

  @Override
  public void print() {
    System.out.println("me");
  }
}
```

Exemplu

Mentimeter

Interfete functionale

- interfete cu o **singura** metoda (neimplementata/abstracta)
- marcate prin adnotarea @FunctionalInterface
- pot fi implementate sub forma de expresii lambda

Mentimeter

```java
@FunctionalInterface
interface ArithmeticalOperation<T> {
  T perform(T a, T b);
}

public class Main {
  public static void main(String[] args) {
    ArithmeticalOperation<Integer> integerAddition = Integer::sum; // (a, b) -> a + b
    ArithmeticalOperation<Double> doublePower = Math::pow; // (a, b) -> Math.pow(a, b)

    System.out.println(integerAddition.perform( a: 1,  b: 2));
    System.out.println(doublePower.perform( a: 10.0,  b: 3.0));
  }
}
```

Exemplu

Mentimeter

Consumer<T>

- functie care primeste un parametru de tip T
- si **NU** intoarce nimic
- Bullet 3

Mentimeter

```java
public class Main {
  public static void main(String[] args) {
    Consumer<Integer> passed = grade -> {
      if (grade > 5) {
        System.out.println("Yay, I passed the exam");
      } else {
        System.out.println("Nope, I failed it");
      }
    };
    List<Integer> grades = IntStream.range(1, 10).boxed().toList();
    grades.forEach(passed);
  }
}
```

Exemplu Consumer

Mentimeter

```java
public class Main {
  public static void main(String[] args) {
    Consumer<Integer> print = grade -> {
      System.out.print("My grade is " + grade + " and I ");
    };
    Consumer<Integer> passed = grade -> {
      if (grade > 5) {
        System.out.println("passed the exam");
      } else {
        System.out.println("failed it");
      }
    };
    List<Integer> grades = IntStream.range(1, 10).boxed().toList();
    grades.forEach(grade -> print.andThen(passed).accept(grade));
  }
}
```

Exemplu andThen

Mentimeter

Predicate<T>

- functie care primeste un parametru de tip T
- si intoarce un boolean

Mentimeter

```java
Predicate<Integer> isAboveAverage = grade -> grade >= 5;
Consumer<Integer> printPassed = grade -> System.out.println("I passed the exam with " + grade);
Consumer<Integer> printFailed = grade -> System.out.println("I failed the exam with " + grade);
List<Integer> grades = IntStream.range(1, 10).boxed().toList();

grades.forEach(grade -> {
  if (isAboveAverage.test(grade)) {
    printPassed.accept(grade);
  } else {
    printFailed.accept(grade);
  }
});
```

Exemplu Predicate

Mentimeter

```java
public class Main {
  public static void main(String[] args) {
    Predicate<Integer> isAboveAverage = grade -> grade >= 5;
    Predicate<Integer> isOdd = grade -> grade % 2 == 1;
    List<Integer> grades = IntStream.range(1, 10).boxed().toList();

    System.out.println(grades.stream().filter(isAboveAverage.and(isOdd.negate())).toList());
  }
}
```

Exemplu and/negate

Mentimeter

BiConsumer<T, U>/BiPredicate<T, U>

- la fel ca si Consumer<T>/Predicate<T>
- daar primeste 2 argumente, nu numai unul

Mentimeter

Function<T, R>

- functie care primeste un parametru de tip T
- si intoarce un rezultat de tip R

Mentimeter

```java
public static void main(String[] args) {
  Function<Integer, String> gradeToString = grade -> "My grade is " + grade;
  List<Integer> grades = List.of(1, 5, 10);
  System.out.println(grades.stream().map(gradeToString).toList());
}
```

Exemplu Function

2

Mentimeter

```java
public static void main(String[] args) {
  Function<Double, Integer> round = value -> Math.toIntExact(Math.round(value));
  Function<Integer, Boolean> isAboveAverage = value -> value >= 5;

  Consumer<Boolean> print = result -> {
    if (result) {
      System.out.println("passed");
    } else {
      System.out.println("failed");
    }
  };
  List<Double> grades = List.of(1.5, 5.8, 10.0);
  Stream<Boolean> results = grades.stream().map(round.andThen(isAboveAverage));
  results.forEach(print);
}
```

Exemplu combo

Mentimeter

```java
public static void main(String[] args) {
  Function<Double, Integer> round = value -> Math.toIntExact(Math.round(value));
  Function<Integer, Boolean> isAboveAverage = value -> value >= 5;

  Consumer<Boolean> print = result -> {
    if (result) {
      System.out.println("passed");
    } else {
      System.out.println("failed");
    }
  };
  List<Double> grades = List.of(1.5, 5.8, 10.0);
  Stream<Boolean> results = grades.stream().map(isAboveAverage.compose(round));
  results.forEach(print);
}
```

Exemplu compose

Mentimeter

Alte chestii

- exista BiFunction<T, U, R> (are 2 argumente si nu unul)
- Function.identity() functie identitate

Mentimeter

Supplier<T>

- nu primeste niciun parametru
- genereaza un rezultat de tip T
- se poate folosi metoda get pentru a genera o valoare

Mentimeter

```java
public class Main {

    public static void main(String[] args) {
        Supplier<Double> generateValue = Math::random;

        for (int i = 0; i < 10; i++) {
            System.out.println(generateValue.get());
        }
    }
}
```

Exemplu Supplier

1

Mentimeter

Streams

- programare functionala
- evaluare lenesa (posibil unbounded)
- consumabile (elementele unui stream sunt vizitate o data cat timp exista streamul)

Mentimeter

Stream pipeline

- o sursa si o secventa de operatii pe sursa
- operatii terminale (ex. forEach, sum) vs operatii intermediare (intorc un nou stream)

1 1 1

Mentimeter

stream() vs parallelStream()

- stream() - executare secventiala in timp
- parallelStream() - executare in paralel pe mai multe core-uri

Mentimeter

```java
public static void main(String[] args) {
  System.out.println(IntStream.range(1, 15).filter(x -> x % 5 == 2).map(x -> x / 2));
  Stream.of(1, 2, 1000, -89).filter(x -> x > 0).findFirst().ifPresent(System.out::println);
}
```

Exemple streams

Mentimeter

Map

- conversia datelor dintr-un Stream prin schimbare de tip/valoare
- to stream: stream()
- from stream: .collect(Collectors.toList()), .collect(Collectors.toSet()), etc

Mentimeter

Filter

- asemanator cu map
- doar ca filtreaza elementele care nu satisfac conditia
- primeste o functie lambda care intoarce un bool

Mentimeter

Reduce

- operatie terminala (aplicatia operatia pe **tot** streamul)
- ex: toArray(), toList()
- accepta 2 argumente: element identitate (sau acumulator) + element

Mentimeter

```java
public static void main(String[] args) {
  String[] words = {"of", "azi", "e", "vineri"};
  System.out.println(Arrays.stream(words).reduce( identity: "", (acc, elem) -> acc + elem));

  int[] values = {1, 200, 300, -1000};
  System.out.println(Arrays.stream(values).reduce(values[0], Math::min));
  // System.out.println(Arrays.stream(values).reduce(values[0], (prevMin, elem) -> Math.min(prevMin, elem)));
}
```

Exemple reduce

1

Mentimeter

flatMap

- functia data poate intoarce mai multe valori (one to many)
- aplica operatia + flatten intr-un singur stream care e intors

Mentimeter

```
String[] words = {"Azi e vineri", "Of e deadline"};
Arrays.stream(words).flatMap(sentence -> Stream.of(sentence.split( regex: " ")))
    .forEach(System.out::println);

HashMap<String, List<Integer>> occurences = new HashMap<>();
occurences.put("ala", List.of(30, 1, 100));
occurences.put("bala", List.of(100, 9, 30));
occurences.put("portocala", List.of(10, 3, 10));

String[] products = {"ala", "portocala"};
Arrays.stream(products).flatMap(product -> Stream.of(occurences.get(product)))
    .forEach(System.out::println);
```

Exemple flatmap



Mentimeter

Super important

functia aplicata cu flatMap
trebuie sa intoarca un stream

Mentimeter

groupingBy

- grupeaza elementele dupa o anumita conditie
- intoarce un Map<T, List<U>>

Mentimeter

```java
public static void main(String[] args) {
  List<String> sentences = List.of("Lab", "POO", "vineri seara", "streamss", "sss");
  Map<Character, List<String>> result = sentences.stream()
      .collect(Collectors.groupingBy(sentence -> sentence.charAt(0)));
  System.out.println(result.get('s'));
}
```

Exemplu groupingBy



Am o lista de cuvinte words, cum pot sa fac un hashmap de frecvente?

Mentimeter

flatmap ✓	map ✓	groupingBy ✓

The correct answer is:
words.groupingBy(Function.identity())

3

Leaderboard

Mentimeter

1000 p  Simba

1000 p  Nemo

1000 p  pizza stream

3

Mentimeter

Keywordul var

- poate fi folosit **numai in interiorul blocurilor de cod**
- **nu** poate fi folosit in declararea campurilor/semnatura metodelor
- se poate folosi numai daca se si initializeaza variabila

Mentimeter

```
var bueno :String = sentences.get(0);
var nope;
```

Exemple var

3